Department of Artificial Intelligence

Course Title: Programing of AI

LAB MANUAL

Submitted To:

Mr. Samraiz

Submitted By:

Ayesha Imran 2023-BS-AI-057

Table of Contents

LAB 01: (Basics)

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal).

[1]:
```
pin, balance, tries = "1234", 10000, 0
while tries < 3:
    if input("Enter PIN: ") == pin:
        amt = float(input("Withdraw: "))
        if amt*1.02 <= balance:
            balance -= amt*1.02
            print("Done! Balance:", balance)
        else:
            print("No funds.")
        break
    else:
        tries += 1
        print("Wrong PIN,", 3-tries, "tries left")
else:
    print("Card Blocked!")
```

```
Enter PIN:  123
Wrong PIN, 2 tries left
Enter PIN:  1234
Withdraw:  1200
Done! Balance: 8776.0
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

[3]:
```python
# Grocery Store Inventory System

store = {
    "apple": {"price": 50, "stock": 20},
    "banana": {"price": 10, "stock": 50},
    "milk": {"price": 100, "stock": 10}
}

cart, total = {}, 0

while True:
    item = input("Enter item (or 'done'): ").lower()
    if item == "done": break
    if item in store:
        qty = int(input("Enter quantity: "))
        if qty <= store[item]["stock"]:
            store[item]["stock"] -= qty
            cost = store[item]["price"] * qty
            cart[item] = cart.get(item, 0) + cost
            total += cost
        else:
            print("❌ Not enough stock.")
    else:
        print("❌ Item not found.")

# Apply discount
if total >= 1000: disc = 0.20
elif total >= 500: disc = 0.15
elif total >= 200: disc = 0.10
else: disc = 0

discount = total * disc
tax = (total - discount) * 0.05
final = total - discount + tax

print("\n🛒 Final Bill:")
for i, cost in cart.items():
    print(f"{i} : {cost}")
print(f"Total: {total}, Discount: {discount}, Tax: {tax}, Final: {final}")
```

```
Enter item (or 'done'):  apple
Enter quantity:  12
Enter item (or 'done'):  done

🛒 Final Bill:
apple : 600
Total: 600, Discount: 90.0, Tax: 25.5, Final: 535.5
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).
Compute weighted average.
Assign grade (A/B/C/D/Fail).
Also, display whether the student is eligible for scholarship (≥80% weighted).

[5]:
```
# Q3. Student Grading with Subject Weightage

weights = (0.2, 0.25, 0.15, 0.25, 0.15)  # weightage for 5 subjects
marks = [float(input(f"Marks for subject {i+1}: ")) for i in range(5)]

# Weighted average
weighted_avg = sum(m * w for m, w in zip(marks, weights))

# Grade assignment
if weighted_avg >= 90: grade = "A"
elif weighted_avg >= 75: grade = "B"
elif weighted_avg >= 60: grade = "C"
elif weighted_avg >= 50: grade = "D"
else: grade = "Fail"

print(f"\nWeighted %: {weighted_avg:.2f}")
print(f"Grade: {grade}")
print("Scholarship Eligible 🎓" if weighted_avg >= 80 else "Not Eligible")
```

```
Marks for subject 1:  90
Marks for subject 2:  45
Marks for subject 3:  90
Marks for subject 4:  67
Marks for subject 5:  89

Weighted %: 72.85
Grade: C
Not Eligible
```

Q4. BMI + Health Recommendation

Extend BMI program:
Take weight, height, and age.
Compute BMI.
Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

[6]:
```
# Q4. BMI + Health Recommendation

w = float(input("Weight (kg): "))
h = float(input("Height (m): "))
age = int(input("Age: "))

bmi = w / (h*h)
print(f"\nBMI: {bmi:.2f}")

if bmi < 18.5:
    msg = "Underweight → Eat protein-rich food & light exercise."
elif bmi < 24.9:
    msg = "Normal → Maintain balanced diet & regular activity."
elif bmi < 29.9:
    msg = "Overweight → Control diet, add cardio exercises."
else:
    msg = "Obese → Strict diet plan + daily workout."

# Age-based note
if age < 18: note = " (Focus on growth, avoid harsh dieting.)"
elif age > 50: note = " (Add walking & regular health checkups.)"
else: note = ""

print("Suggestion:", msg + note)
```

```
Weight (kg):  12
Height (m):  6
Age:  21

BMI: 0.33
Suggestion: Underweight → Eat protein-rich food & light exercise.
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

[7]:
```
# Q5. Telecom Prepaid Recharge System

rates = {"call": 2, "sms": 1, "data": 5}  # per min, per SMS, per MB
balance = float(input("Enter recharge amount: "))
init_bal = balance

if balance > 1000:  # bonus
    balance += 100
    print("🎁 Bonus 100 added! New balance:", balance)

while True:
    use = input("\nEnter usage (call/sms/data/exit): ").lower()
    if use == "exit": break
    if use in rates:
        qty = float(input("Enter units: "))
        cost = qty * rates[use]
        if cost <= balance:
            balance -= cost
            print(f"Used {qty} {use}, Cost={cost}, Balance={balance}")
            if balance < 0.2 * init_bal:
                print("⚠️ Low Balance! Please recharge.")
        else:
            print("❌ Not enough balance.")
    else:
        print("❌ Invalid option.")
```

```
Enter recharge amount:  2200
🎁 Bonus 100 added! New balance: 2300.0

Enter usage (call/sms/data/exit):  call
Enter units:  45
Used 45.0 call, Cost=90.0, Balance=2210.0

Enter usage (call/sms/data/exit):  exit
```

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

[8]:
```
# Q6. E-Commerce Discount Engine

bill = float(input("Enter bill amount: "))
mem = input("Membership (Regular/Gold/Platinum): ").lower()

# Membership discount
disc = 0
if mem == "gold": disc += 0.10
elif mem == "platinum": disc += 0.15

# Extra discount for big bill
if bill > 10000: disc += 0.05

discount = bill * disc
subtotal = bill - discount
tax = subtotal * 0.05
final = subtotal + tax

print("\n Receipt")
print(f"Bill: {bill}")
print(f"Discount ({disc*100}%): -{discount}")
print(f"Subtotal: {subtotal}")
print(f"Tax (5%): +{tax}")
print(f"Final Payable: {final}")
```

```
Enter bill amount:  2400
Membership (Regular/Gold/Platinum):  gold

 Receipt
Bill: 2400.0
Discount (10.0%): -240.0
Subtotal: 2160.0
Tax (5%): +108.0
Final Payable: 2268.0
```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest “Admit in ICU.”

If urgent → suggest “Immediate doctor attention.”

Use nested conditions.

[9]:
```
# Q7. Hospital Emergency Unit Triage

temp = float(input("Temperature (°C): "))
pulse = int(input("Pulse rate: "))
oxy = int(input("Oxygen level %: "))
bp = int(input("Blood Pressure (systolic): "))

if temp > 40 or oxy < 85 or bp < 80:
    status, advice = "Critical", "Admit in ICU!"
elif temp > 38 or oxy < 90 or pulse > 120 or bp < 90:
    status, advice = "Urgent", "Immediate doctor attention!"
elif temp > 37.5 or pulse > 100 or oxy < 94:
    status, advice = "Observation", "Keep under monitoring."
else:
    status, advice = "Stable", "Normal care, no emergency."

print(f"\nStatus: {status}\nAdvice: {advice}")
```

```
Temperature (°C):  34
Pulse rate:  98
Oxygen level %:  34
Blood Pressure (systolic):  34

Status: Critical
Advice: Admit in ICU!
```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."

[10]:
```
# Q8. Online Examination Grader

total = int(input("Total Marks: "))
obt = float(input("Obtained Marks: "))
wrong = int(input("Wrong Answers: "))

# Negative marking
obt -= wrong * 0.25
perc = (obt / total) * 100

# Division
if perc >= 60: div = "First"
elif perc >= 45: div = "Second"
elif perc >= 33: div = "Third"
else: div = "Fail"

print(f"\nFinal Marks: {obt:.2f}/{total}")
print(f"Percentage: {perc:.2f}%")
print(f"Division: {div}")

if perc >= 80: print("Scholarship Eligible 🎓")
elif div == "Fail": print("❌ Repeat Exam")
```

```
Total Marks:  980
Obtained Marks:  900
Wrong Answers:  76

Final Marks: 881.00/980
Percentage: 89.90%
Division: First
Scholarship Eligible 🎓
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

[11]:
```python
# Q9. Movie Ticket Booking

prices = {"normal": 500, "3d": 700, "imax": 1000}

age = int(input("Enter age: "))
show = int(input("Showtime (24hr e.g. 21 for 9pm): "))
typ = input("Ticket type (Normal/3D/IMAX): ").lower()

if age < 12 and show >= 22:
    print("❌ Children under 12 not allowed for late-night shows.")
else:
    price = prices.get(typ, 500)

    if age >= 60: price *= 0.7          # 30% discount
    if 18 <= show <= 22: price *= 1.10  # peak hour

    print(f"Final Ticket Price: {price}")
```

```
Enter age:  20
Showtime (24hr e.g. 21 for 9pm):  5
Ticket type (Normal/3D/IMAX):  3d
Final Ticket Price: 700
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend “light formal suit.”

[12]:
```
# Q10. Weather-based Outfit + Travel Suggestion

temp = float(input("Temperature (°C): "))
weather = input("Weather (Rainy/Sunny/Cold): ").lower()
event = input("Event (Casual/Business/Party): ").lower()

if event == "business" and temp > 35:
    outfit = "Light formal suit"
elif event == "casual":
    outfit = "T-shirt & jeans"
elif event == "party":
    outfit = "Trendy outfit with accessories"
else:
    outfit = "Formal wear"

if weather == "rainy": outfit += " + Umbrella"
elif weather == "cold": outfit += " + Gloves"
else: outfit += " + Sunglasses"

print("\nSuggestion:", outfit)
```

```
Temperature (°C):  43
Weather (Rainy/Sunny/Cold):  cold
Event (Casual/Business/Party):  party

Suggestion: Trendy outfit with accessories + Gloves
```

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.

[13]:
```
# Q11. ATM Withdrawal with Note Counting + Receipt

balance = 20000  # initial balance
amt = int(input("Enter withdrawal amount: "))

if amt > balance:
    print("❌ Insufficient Balance")
else:
    notes = {}
    for n in [1000, 500, 100, 50, 10, 1]:
        notes[n], amt = divmod(amt, n)

    withdraw = sum(k*v for k,v in notes.items())
    charge = withdraw * 0.005
    total = withdraw + charge
    balance -= total

    print("\n🧾 Receipt")
    print("Notes:", {k:v for k,v in notes.items() if v>0})
    print(f"Withdrawal: {withdraw}")
    print(f"Service Charge (0.5%): {charge:.2f}")
    print(f"Remaining Balance: {balance:.2f}")
```

```
Enter withdrawal amount:  4500

🧾 Receipt
Notes: {1000: 4, 500: 1}
Withdrawal: 4500
Service Charge (0.5%): 22.50
Remaining Balance: 15477.50
```

Q12. Library Fine Calculator with Membership Levels

Input days late.

Apply fines (10, 20, 50/day).

If days > 30 → “Membership Cancelled.”

If user is Premium member, reduce fine by 50%.

Print detailed fine slip.

[14]:
```python
# Q12. Library Fine Calculator

days = int(input("Days late: "))
member = input("Membership (Normal/Premium): ").lower()

if days > 30:
    print("❌ Membership Cancelled.")
else:
    if days <= 10: fine = days * 10
    elif days <= 20: fine = days * 20
    else: fine = days * 50

    if member == "premium": fine *= 0.5
    print(f"\n🧾 Fine Slip\nDays Late: {days}\nFine: {fine}")
```

```
Days late:  3
Membership (Normal/Premium):  normal

🧾 Fine Slip
Days Late: 3
Fine: 30
```

Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).

Print max, min, average.

Print months above average.

Use loop + condition to print “Growth” or “Decline” compared to last month.

[15]:
```
# Q13. Sales Analysis Dashboard

sales = [120, 150, 100, 200, 170, 180, 220, 190, 250, 210, 230, 300]

mx, mn, avg = max(sales), min(sales), sum(sales)/12
print(f"Max={mx}, Min={mn}, Avg={avg:.2f}")

print("Above Avg Months:", [m+1 for m,v in enumerate(sales) if v>avg])

for i in range(1, 12):
    print(f"Month {i+1}: {'Growth' if sales[i]>sales[i-1] else 'Decline'}")
```

```
Max=300, Min=100, Avg=193.33
Above Avg Months: [4, 7, 9, 10, 11, 12]
Month 2: Growth
Month 3: Decline
Month 4: Growth
Month 5: Decline
Month 6: Growth
Month 7: Growth
Month 8: Decline
Month 9: Growth
Month 10: Decline
Month 11: Growth
Month 12: Growth
```

Q14. Multiplication Puzzle with Random Blanks

Generate a multiplication table (1–10).

Randomly replace 3 results with "??".

Ask user to guess values.

Track score.

Give performance remark (Excellent/Good/Try Again).

```
[16]: # Q14. Multiplication Puzzle

import random
score = 0
table = [(i,j,i*j) for i in range(1,11) for j in range(1,11)]
blanks = random.sample(range(100), 3)

for k,(a,b,res) in enumerate(table):
    if k in blanks:
        ans = int(input(f"{a} x {b} = ?? "))
        if ans == res: score += 1
    else:
        print(f"{a} x {b} = {res}")

print("Score:", score, "/3")
print("Remark:", "Excellent" if score==3 else "Good" if score==2 else "Try Again")
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = ??  5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).

If weak → ask user to re-enter.

Allow max 3 attempts.

If still weak → “Account Locked.”

```
[17]: # Q15. Password Strength

import re
def strong(p):
    return (len(p)>=8 and re.search(r"[A-Z]",p) and re.search(r"[a-z]",p)
            and re.search(r"\d",p) and re.search(r"\W",p))

tries=0
while tries<3:
    pw=input("Enter password: ")
    if strong(pw): print("✅ Strong Password"); break
    else: print("❌ Weak, try again"); tries+=1
else: print("🚫 Account Locked")
```

```
Enter password:  1234
❌ Weak, try again
Enter password:  Ayesha4657
✅ Strong Password
```

```
[ ]:
```

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):

Apply slab rates (5/7/10 Rs).

If late_payment → add 15% penalty.

Return bill breakdown.

[18]:
```
# Q16. Electricity Bill

def calculate_bill(units, late=False):
    if units<=100: bill=units*5
    elif units<=200: bill=100*5+(units-100)*7
    else: bill=100*5+100*7+(units-200)*10
    penalty=bill*0.15 if late else 0
    return f"Units:{units}, Bill:{bill}, Penalty:{penalty}, Total:{bill+penalty}"

print(calculate_bill(250, late=True))
```

```
Units:250, Bill:1700, Penalty:255.0, Total:1955.0
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → “Losing.”

If run rate > 7 and wickets < 5 → “Winning.”

Else → “Balanced.”

[19]:
```
# Q17. Cricket Match Predictor

def predict_score(runs, balls, wickets, overs_left):
    sr=(runs/balls)*100; rr=runs/(50-overs_left)
    if rr<5 and wickets>6: return "Losing"
    elif rr>7 and wickets<5: return "Winning"
    else: return "Balanced"

print(predict_score(200, 180, 7, 10))
```

```
Balanced
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):

Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.

If days > 7 → 20% discount.

If member → extra 10% discount.

Add 12% tax.

Return bill slip.

```
[20]: # Q18. Hotel Booking

def book_room(t, days, member):
    rates={"standard":2000,"deluxe":4000,"suite":6000}
    cost=days*rates[t.lower()]
    if days>7: cost*=0.8
    if member: cost*=0.9
    tax=cost*0.12
    return f"Days:{days}, Cost:{cost}, Tax:{tax}, Total:{cost+tax}"

print(book_room("Suite", 8, True))
```

```
Days:8, Cost:34560.0, Tax:4147.2, Total:38707.2
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):

Economy: 10/unit, Business: 25/unit, First: 40/unit.

Add fuel surcharge 500.

If international → add 18% tax.

If booking in advance (>30 days) → 10% discount.

```
[21]: # Q19. Flight Fare Calculator

def calculate_fare(dist, ctype, intl=False, advance=False):
    rates={"economy":10,"business":25,"first":40}
    fare=dist*rates[ctype.lower()]+500
    if intl: fare*=1.18
    if advance: fare*=0.9
    return f"Fare: {fare}"

print(calculate_fare(800, "Business", True, True))
```

```
Fare: 21771.0
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):

Items stored in dict with stock + price.

If sufficient → dispense + return change.

Update wallet balance.

If stock = 0 → suggest alternative item.

[22]:
```
# Q20. Smart Vending Machine

items={"chips":{"price":50,"stock":5},
       "soda":{"price":80,"stock":3},
       "candy":{"price":30,"stock":0}}

def vending_machine(it, paid, wallet):
    if it not in items: return "❌ Item not found"
    if items[it]["stock"]<=0: return f"❌ Out of stock. Try another."
    if paid+wallet < items[it]["price"]: return "❌ Not enough balance"

    cost=items[it]["price"]; items[it]["stock"]-=1
    change=paid+wallet-cost; return f"Dispensed {it}, Change={change}"

print(vending_machine("soda", 50, 40))
```

```
Dispensed soda, Change=10
```

LAB 02: (OOP in Python)

Question # 1: Create a `Bank Account` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

```python
class BankAccount:
    def __init__(self, number, name, balance=0):
        self.number = number
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough balance!")
        else:
            self.balance -= amount
            print("Withdrawn:", amount)

    def show_balance(self):
        print("Account Holder:", self.name)
        print("Balance:", self.balance)


# --- Test Code ---
acc1 = BankAccount("A-1001", "Ayesha", 1000)
acc1.show_balance()
acc1.deposit(500)
acc1.withdraw(200)
acc1.withdraw(2000)
acc1.show_balance()
```

```
Account Holder: Ayesha
Balance: 1000
Deposited: 500
Withdrawn: 200
Not enough balance!
Account Holder: Ayesha
Balance: 1300
```

Question # 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.available = True


class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print("Book added:", book.title)

    def search(self, title):
        for b in self.books:
            if title.lower() in b.title.lower():
                print(f"{b.title} by {b.author} | Available: {b.available}")

    def checkout(self, title):
        for b in self.books:
            if b.title.lower() == title.lower() and b.available:
                b.available = False
                print("Book borrowed:", b.title)
                return
        print("Book not available!")

    def return_book(self, title):
        for b in self.books:
            if b.title.lower() == title.lower():
                b.available = True
                print("Book returned:", b.title)
                return
        print("Book not found!")
```

```python
# --- Test Code ---
lib = Library()
b1 = Book("Python Basics", "John")
b2 = Book("OOP in Python", "Ayesha")

lib.add_book(b1)
lib.add_book(b2)

lib.search("Python")
lib.checkout("Python Basics")
lib.search("Python")
lib.return_book("Python Basics")
lib.search("Python")
```

```
Book added: Python Basics
Book added: OOP in Python
Python Basics by John | Available: True
OOP in Python by Ayesha | Available: True
Book borrowed: Python Basics
Python Basics by John | Available: False
OOP in Python by Ayesha | Available: True
Book returned: Python Basics
Python Basics by John | Available: True
OOP in Python by Ayesha | Available: True
```

Question # 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)

    def letter_grade(self):
        avg = self.average()
        if avg >= 90:
            return "A"
        elif avg >= 80:
            return "B"
        elif avg >= 70:
            return "C"
        elif avg >= 60:
            return "D"
        else:
            return "F"

    def show_info(self):
        print("Student:", self.name)
        print("Average:", round(self.average(), 2))
        print("Grade:", self.letter_grade())


# --- Test Code ---
s1 = Student("S-101", "Ayesha")
s1.add_grade(85)
s1.add_grade(90)
s1.add_grade(80)
s1.show_info()
```

```
Student: Ayesha
Average: 85.0
Grade: B
```

Question # 4: Design a `Menu Item` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

```
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category


class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)
        print("Added:", item.name)

    def total_cost(self):
        return sum(i.price for i in self.items)

    def apply_discount(self, percent):
        total = self.total_cost()
        discount = total * (percent / 100)
        return total - discount

    def generate_receipt(self):
        print("\n Receipt:")
        for i in self.items:
            print(f"{i.name} - Rs.{i.price}")
        total = self.total_cost()
        print("Total:", total)
        print("After 10% Discount:", self.apply_discount(10))
        print("With 5% Tax:", round(self.apply_discount(10) * 1.05, 2))


# --- Test Code ---
item1 = MenuItem("Burger", 250, "Fast Food")
item2 = MenuItem("Pizza", 800, "Fast Food")
item3 = MenuItem("Juice", 150, "Drink")

order1 = Order()
order1.add_item(item1)
order1.add_item(item2)
order1.add_item(item3)
order1.generate_receipt()
```

```
Added: Burger
Added: Pizza
Added: Juice

 Receipt:
Burger - Rs.250
Pizza - Rs.800
Juice - Rs.150
Total: 1200
After 10% Discount: 1080.0
With 5% Tax: 1134.0
```

Question # 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

```
class Vehicle:
    def __init__(self, plate, model, rent_per_day):
        self.plate = plate
        self.model = model
        self.rent_per_day = rent_per_day
        self.available = True


class Car(Vehicle):
    pass

class Motorcycle(Vehicle):
    pass

class Truck(Vehicle):
    pass


class RentalSystem:
    def __init__(self):
        self.vehicles = []

    def add_vehicle(self, vehicle):
        self.vehicles.append(vehicle)
        print("Vehicle added:", vehicle.model)

    def rent_vehicle(self, plate, days):
        for v in self.vehicles:
            if v.plate == plate and v.available:
                v.available = False
                cost = v.rent_per_day * days
                print(f"{v.model} rented for {days} days. Total cost: Rs.{cost}")
                return
        print("Vehicle not available!")

    def return_vehicle(self, plate):
        for v in self.vehicles:
            if v.plate == plate:
                v.available = True
                print(f"{v.model} returned.")
                return
        print("Invalid plate number!")
```

```
# --- Test Code ---
car1 = Car("ABC-101", "Toyota Corolla", 5000)
bike1 = Motorcycle("XYZ-202", "Honda 125", 1500)
truck1 = Truck("TRK-303", "Hino Truck", 8000)

system = RentalSystem()
system.add_vehicle(car1)
system.add_vehicle(bike1)
system.add_vehicle(truck1)

system.rent_vehicle("ABC-101", 3)
system.return_vehicle("ABC-101")
system.rent_vehicle("TRK-303", 2)
```

```
Vehicle added: Toyota Corolla
Vehicle added: Honda 125
Vehicle added: Hino Truck
Toyota Corolla rented for 3 days. Total cost: Rs.15000
Toyota Corolla returned.
Hino Truck rented for 2 days. Total cost: Rs.16000
```

Question # 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

```
class Employee:
    def __init__(self, emp_id, name, position, salary):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary

    def monthly_salary(self):
        print(f"Monthly Salary of {self.name}: Rs.{self.salary}")

    def annual_salary(self):
        print(f"Annual Salary of {self.name}: Rs.{self.salary * 12}")

    def apply_raise(self, percent):
        increase = self.salary * (percent / 100)
        self.salary += increase
        print(f"Salary raised by {percent}%. New Salary: Rs.{self.salary}")


# --- Test Code ---
emp1 = Employee("E-101", "Ayesha", "Manager", 60000)
emp1.monthly_salary()
emp1.annual_salary()
emp1.apply_raise(10)
emp1.monthly_salary()
```

```
Monthly Salary of Ayesha: Rs.60000
Annual Salary of Ayesha: Rs.720000
Salary raised by 10%. New Salary: Rs.66000.0
Monthly Salary of Ayesha: Rs.66000.0
```

Question # 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `Shopping Cart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price


class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_product(self, product):
        self.items.append(product)
        print(f"{product.name} added to cart.")

    def total_price(self):
        total = sum(item.price for item in self.items)
        print(f"Total Price: Rs.{total}")

    def show_items(self):
        print("\nItems in Cart:")
        for item in self.items:
            print(f"- {item.name}: Rs.{item.price}")


# --- Test Code ---
p1 = Product("Shirt", 1200)
p2 = Product("Shoes", 2500)
p3 = Product("Watch", 1800)

cart = ShoppingCart()
cart.add_product(p1)
cart.add_product(p2)
cart.add_product(p3)
cart.show_items()
cart.total_price()
```

```
Shirt added to cart.
Shoes added to cart.
Watch added to cart.

Items in Cart:
- Shirt: Rs.1200
- Shoes: Rs.2500
- Watch: Rs.1800
Total Price: Rs.5500
```

Question # 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.available = True


class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print(f"Book '{book.title}' added to library.")

    def show_books(self):
        print("\nBooks in Library:")
        for b in self.books:
            status = "Available" if b.available else "Not Available"
            print(f"- {b.title} by {b.author} ({status})")

    def borrow_book(self, title):
        for b in self.books:
            if b.title == title and b.available:
                b.available = False
                print(f"You borrowed '{b.title}'.")
                return
        print("Sorry, book not available.")

    def return_book(self, title):
        for b in self.books:
            if b.title == title and not b.available:
                b.available = True
                print(f"Book '{b.title}' returned successfully.")
                return
        print("This book was not borrowed.")


# --- Test Code ---
b1 = Book("Python Basics", "John Smith")
b2 = Book("AI for Beginners", "Ayesha Imran")

lib = Library()
lib.add_book(b1)
lib.add_book(b2)
lib.show_books()

lib.borrow_book("Python Basics")
lib.show_books()

lib.return_book("Python Basics")
lib.show_books()
```

```
Book 'Python Basics' added to library.
Book 'AI for Beginners' added to library.

Books in Library:
- Python Basics by John Smith (Available)
- AI for Beginners by Ayesha Imran (Available)
You borrowed 'Python Basics'.

Books in Library:
- Python Basics by John Smith (Not Available)
- AI for Beginners by Ayesha Imran (Available)
Book 'Python Basics' returned successfully.

Books in Library:
- Python Basics by John Smith (Available)
- AI for Beginners by Ayesha Imran (Available)
```

Question # 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
        self.marks = []

    def add_marks(self, mark):
        self.marks.append(mark)

    def average(self):
        if len(self.marks) == 0:
            return 0
        return sum(self.marks) / len(self.marks)

    def grade(self):
        avg = self.average()
        if avg >= 80:
            return "A"
        elif avg >= 60:
            return "B"
        elif avg >= 40:
            return "C"
        else:
            return "F"

    def show_result(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self.roll_no}")
        print(f"Average Marks: {self.average():.2f}")
        print(f"Grade: {self.grade()}")


# --- Test Code ---
s1 = Student("Ayesha", 101)
s1.add_marks(85)
s1.add_marks(75)
s1.add_marks(90)

s1.show_result()
```

```
Name: Ayesha
Roll No: 101
Average Marks: 83.33
Grade: A
```

Question # 10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

```
import math

class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height


# --- Test Code ---
c = Circle(5)
r = Rectangle(4, 6)
t = Triangle(3, 7)

print("Circle Area:", round(c.area(), 2))
print("Rectangle Area:", r.area())
print("Triangle Area:", t.area())
```

```
Circle Area: 78.54
Rectangle Area: 24
Triangle Area: 10.5
```

Question # 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

```
0]: class Product:
        def __init__(self, pid, name, quantity, price, supplier):
            self.pid = pid
            self.name = name
            self.quantity = quantity
            self.price = price
            self.supplier = supplier


    class Inventory:
        def __init__(self):
            self.products = []

        def add_product(self, product):
            self.products.append(product)
            print(f"{product.name} added to inventory.")

        def update_quantity(self, pid, qty):
            for p in self.products:
                if p.pid == pid:
                    p.quantity += qty
                    print(f"{p.name} quantity updated to {p.quantity}")
                    return
            print("Product not found!")

        def low_stock_alert(self):
            print("\n⚠️ Low Stock Products:")
            for p in self.products:
                if p.quantity < 5:
                    print(f"- {p.name} (Qty: {p.quantity})")


    # --- Test Code ---
    i = Inventory()
    p1 = Product(1, "Laptop", 4, 80000, "Dell")
    p2 = Product(2, "Mouse", 10, 500, "Logitech")

    i.add_product(p1)
    i.add_product(p2)

    i.update_quantity(1, 2)
    i.low_stock_alert()
```

```
Laptop added to inventory.
Mouse added to inventory.
Laptop quantity updated to 6

⚠️ Low Stock Products:
```

Question # 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

```python
class Product:
    def __init__(self, pid, name, quantity, price, supplier):
        self.pid = pid
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier


class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)
        print(f"{product.name} added to inventory.")

    def update_quantity(self, pid, qty):
        for p in self.products:
            if p.pid == pid:
                p.quantity += qty
                print(f"{p.name} quantity updated to {p.quantity}")
                return
        print("Product not found!")

    def low_stock_alert(self):
        print("\n⚠️ Low Stock Products:")
        for p in self.products:
            if p.quantity < 5:
                print(f"- {p.name} (Qty: {p.quantity})")


# --- Test Code ---
i = Inventory()
p1 = Product(1, "Laptop", 4, 80000, "Dell")
p2 = Product(2, "Mouse", 10, 500, "Logitech")

i.add_product(p1)
i.add_product(p2)

i.update_quantity(1, 2)
i.low_stock_alert()
```

```
Laptop added to inventory.
Mouse added to inventory.
Laptop quantity updated to 6

⚠️ Low Stock Products:
```

Question # 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

```
class Movie:
    def __init__(self, title, genre, duration, showtime):
        self.title = title
        self.genre = genre
        self.duration = duration
        self.showtime = showtime


class Theater:
    def __init__(self, name, seats):
        self.name = name
        self.seats = seats
        self.booked = 0

    def book_seat(self, num):
        if self.booked + num <= self.seats:
            self.booked += num
            print(f"{num} seat(s) booked successfully!")
        else:
            print("Not enough seats available!")

    def available_seats(self):
        return self.seats - self.booked


# --- Test Code ---
m1 = Movie("Avengers", "Action", 120, "7:00 PM")
t1 = Theater("Galaxy Cinema", 50)

print(f"🎬 Movie: {m1.title} | Show: {m1.showtime}")
print(f"Available Seats: {t1.available_seats()}")

t1.book_seat(5)
print(f"Seats left: {t1.available_seats()}")
```

```
🎬 Movie: Avengers | Show: 7:00 PM
Available Seats: 50
5 seat(s) booked successfully!
Seats left: 45
```

Question # 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

```python
class Member:
    def __init__(self, name, membership_type):
        self.name = name
        self.membership_type = membership_type
        self.attendance = 0

    def mark_attendance(self):
        self.attendance += 1
        print(f"{self.name}'s attendance marked. Total: {self.attendance}")


class Trainer:
    def __init__(self, name):
        self.name = name
        self.sessions = []

    def schedule_session(self, member):
        self.sessions.append(member)
        print(f"Session scheduled for {member.name} with Trainer {self.name}")


# --- Test Code ---
m1 = Member("Ayesha", "Premium")
m2 = Member("Ali", "Basic")

t1 = Trainer("Hassan")

t1.schedule_session(m1)
t1.schedule_session(m2)

m1.mark_attendance()
m2.mark_attendance()
```

```
Session scheduled for Ayesha with Trainer Hassan
Session scheduled for Ali with Trainer Hassan
Ayesha's attendance marked. Total: 1
Ali's attendance marked. Total: 1
```

Question # 15: Extend the bank account system to include `Savings Account` and `Checking Account` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

```python
class BankAccount:
    def __init__(self, acc_no, holder, balance=0):
        self.acc_no = acc_no
        self.holder = holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited: {amount}, New Balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrawn: {amount}, Remaining: {self.balance}")
        else:
            print("Insufficient balance!")

    def transfer(self, other, amount):
        if amount <= self.balance:
            self.balance -= amount
            other.balance += amount
            print(f"Transferred {amount} to {other.holder}")
        else:
            print("Transfer failed! Not enough balance.")


class SavingsAccount(BankAccount):
    def __init__(self, acc_no, holder, balance=0, interest_rate=0.03):
        super().__init__(acc_no, holder, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"Interest added: {interest}, New Balance: {self.balance}")


class CheckingAccount(BankAccount):
    def __init__(self, acc_no, holder, balance=0):
        super().__init__(acc_no, holder, balance)

    def withdraw(self, amount):
        fee = 10  # small transaction fee
        total = amount + fee
        if total <= self.balance:
            self.balance -= total
            print(f"Withdrawn: {amount} (Fee {fee}), Remaining: {self.balance}")
        else:
            print("Insufficient funds!")
```

```python
# --- Test Code ---
s1 = SavingsAccount(101, "Ayesha", 1000)
c1 = CheckingAccount(102, "Ali", 1500)

s1.add_interest()
s1.transfer(c1, 200)
c1.withdraw(300)
```

```
Interest added: 30.0, New Balance: 1030.0
Transferred 200 to Ali
Withdrawn: 300 (Fee 10), Remaining: 1390
```

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

```python
class Pizza:
    def __init__(self, size, crust, toppings):
        self.size = size
        self.crust = crust
        self.toppings = toppings

    def calculate_price(self):
        base_price = 500 if self.size == "Small" else 800 if self.size == "Medium" else 1100
        topping_price = 50 * len(self.toppings)
        return base_price + topping_price


class Order:
    def __init__(self):
        self.pizzas = []

    def add_pizza(self, pizza):
        self.pizzas.append(pizza)
        print(f"{pizza.size} Pizza added to order!")

    def total_bill(self):
        total = sum(p.calculate_price() for p in self.pizzas)
        print(f"Total Bill: Rs {total}")


# --- Test Code ---
p1 = Pizza("Medium", "Cheese Burst", ["Olives", "Mushroom"])
p2 = Pizza("Large", "Thin Crust", ["Pepperoni", "Cheese", "Capsicum"])

order = Order()
order.add_pizza(p1)
order.add_pizza(p2)
order.total_bill()
```

```
Medium Pizza added to order!
Large Pizza added to order!
Total Bill: Rs 2150
```

Question # 17: Design classes for `Car` (model, year, color, price) and `Car Manufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price

    def show_info(self):
        print(f"{self.year} {self.color} {self.model} – Rs {self.price}")


class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.cars = []

    def produce_car(self, model, year, color, price):
        car = Car(model, year, color, price)
        self.cars.append(car)
        print(f"{self.name} produced a {model}")
        return car

    def show_all_cars(self):
        print(f"\nCars produced by {self.name}:")
        for car in self.cars:
            car.show_info()


# --- Test Code ---
m1 = CarManufacturer("Toyota")
m1.produce_car("Corolla", 2023, "White", 4500000)
m1.produce_car("Yaris", 2022, "Black", 3800000)

m1.show_all_cars()
```

```
Toyota produced a Corolla
Toyota produced a Yaris

Cars produced by Toyota:
2023 White Corolla – Rs 4500000
2022 Black Yaris – Rs 3800000
```

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

```python
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject


class Student:
    def __init__(self, name):
        self.name = name
        self.grades = {}

    def add_grade(self, subject, grade):
        self.grades[subject] = grade

    def average_grade(self):
        if not self.grades:
            return 0
        return sum(self.grades.values()) / len(self.grades)


class Classroom:
    def __init__(self, name, teacher):
        self.name = name
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        print(f"{student.name} added to {self.name}")

    def show_report(self):
        print(f"\nClassroom: {self.name}")
        print(f"Teacher: {self.teacher.name} ({self.teacher.subject})")
        for s in self.students:
            print(f"{s.name} - Average: {s.average_grade():.2f}")


# --- Test Code ---
t1 = Teacher("Sir Ahmed", "Math")
c1 = Classroom("BSCS-1A", t1)

s1 = Student("Ayesha")
s2 = Student("Ali")

s1.add_grade("Math", 90)
s1.add_grade("Science", 85)
s2.add_grade("Math", 70)

c1.add_student(s1)
c1.add_student(s2)
c1.show_report()
```

```
Ayesha added to BSCS-1A
Ali added to BSCS-1A

Classroom: BSCS-1A
Teacher: Sir Ahmed (Math)
Ayesha - Average: 87.50
Ali - Average: 70.00
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

```python
class Passenger:
    def __init__(self, name):
        self.name = name


class Flight:
    def __init__(self, flight_no, destination, capacity):
        self.flight_no = flight_no
        self.destination = destination
        self.capacity = capacity
        self.passengers = []

    def book_passenger(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            print(f"{passenger.name} booked on flight {self.flight_no}")
        else:
            print("Flight full!")

    def show_passengers(self):
        print(f"\nPassengers on flight {self.flight_no}:")
        for p in self.passengers:
            print("-", p.name)


# --- Test Code ---
f1 = Flight("PK101", "Karachi", 3)

p1 = Passenger("Ayesha")
p2 = Passenger("Ali")
p3 = Passenger("Sara")
p4 = Passenger("Bilal")

f1.book_passenger(p1)
f1.book_passenger(p2)
f1.book_passenger(p3)
f1.book_passenger(p4)  # Should say Flight full!

f1.show_passengers()
```

```
Ayesha booked on flight PK101
Ali booked on flight PK101
Sara booked on flight PK101
Flight full!

Passengers on flight PK101:
- Ayesha
- Ali
- Sara
```

Question # 20: Create classes for `Smart Device` (base class) and specific devices like `Smart Light`, `Thermostat`, and `Security Camera`. Implement a `Smart Home` class that can control all devices and create automation routines.

```python
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "OFF"

    def turn_on(self):
        self.status = "ON"
        print(f"{self.name} is ON")

    def turn_off(self):
        self.status = "OFF"
        print(f"{self.name} is OFF")


class SmartLight(SmartDevice):
    pass

class Thermostat(SmartDevice):
    pass

class SecurityCamera(SmartDevice):
    pass


class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        print(f"{device.name} added to SmartHome")

    def turn_all_on(self):
        for d in self.devices:
            d.turn_on()

    def turn_all_off(self):
        for d in self.devices:
            d.turn_off()


# --- Test Code ---
l1 = SmartLight("Living Room Light")
t1 = Thermostat("Main Thermostat")
c1 = SecurityCamera("Front Door Camera")

home = SmartHome()
home.add_device(l1)
home.add_device(t1)
home.add_device(c1)

home.turn_all_on()
home.turn_all_off()
```

```
Living Room Light added to SmartHome
Main Thermostat added to SmartHome
Front Door Camera added to SmartHome
Living Room Light is ON
Main Thermostat is ON
Front Door Camera is ON
Living Room Light is OFF
Main Thermostat is OFF
Front Door Camera is OFF
```

LAB 03: (Numpy)

Question 1: A weather station records temperatures (°C) for 7 days:
[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- Convert the list into a NumPy array
- Find:
    - Average temperature
    - Maximum and minimum temperature
    - Temperature difference (range)

```python
import numpy as np

# The list of temperatures provided
temperatures_list = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

# Convert the list into a numpy array
temperatures_array = np.array(temperatures_list)

# Calculate the metrics
average_temperature = np.mean(temperatures_array)
maximum_temperature = np.max(temperatures_array)
minimum_temperature = np.min(temperatures_array)
temperature_difference = maximum_temperature - minimum_temperature

# Print the results
print(f"Numpy Array: {temperatures_array}")
print(f"Average Temperature: {average_temperature:.2f}°C")
print(f"Maximum Temperature: {maximum_temperature:.2f}°C")
print(f"Minimum Temperature: {minimum_temperature:.2f}°C")
print(f"Temperature Difference (Range): {temperature_difference:.2f}°C")
```

✓ 2.7s

```
Numpy Array: [21.1 23.4 19.8 25.  26.5 24.1 22. ]
Average Temperature: 23.13°C
Maximum Temperature: 26.50°C
Minimum Temperature: 19.80°C
Temperature Difference (Range): 6.70°C
```

Question 2: A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- Create a **4×7 NumPy array**
- Find weekly totals
- Find the highest sale day (index only)
- Find the week with the most sales

Code:

```
import numpy as np

# Data provided
sales_data = [
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11]
]

# Task: Create a 4x7 NumPy array
sales_array = np.array(sales_data)
print(f"Sales Array:\n{sales_array}")
```

✓ 0.0s

```
Sales Array:
[[12 15 14 10  9  8  7]
 [11 12 13  9  5  8  6]
 [ 7  9 12 10 11 13 12]
 [10 11 12  7  8  9 11]]
```

Question 3: Students received marks in a test:
[56, 78, 45, 90, 88, 76, 59, 62]

Task:

- Convert to NumPy array
- Add **5 bonus marks** to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation

Code:

```python
import numpy as np

marks_list = [56, 78, 45, 90, 88, 76, 59, 62]

# Convert to NumPy array
marks_array = np.array(marks_list)
print(f"NumPy array: {marks_array}")

# Add 5 bonus marks
bonus_marks = marks_array + 5
print(f"Marks with bonus: {bonus_marks}")

# Count students scoring > 60
count_above_60 = np.sum(bonus_marks > 60)
print(f"Count above 60: {count_above_60}")

# Find median and standard deviation
median_marks = np.median(bonus_marks)
std_dev_marks = np.std(bonus_marks)
print(f"Median: {median_marks}")
print(f"Standard Deviation: {std_dev_marks}")
```

✓ 0.0s

```
NumPy array: [56 78 45 90 88 76 59 62]
Marks with bonus: [61 83 50 95 93 81 64 67]
Count above 60: 7
Median: 74.0
Standard Deviation: 15.105876340020794
```

Question 4: A smartwatch stores daily step-counts for a user for 30 days.

Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000
- Reshape into a **5×6** matrix
- Find:
    - Weekly average steps
    - Day with maximum steps
    - Number of days user crossed 10,000 steps

Code:

```
import numpy as np

# Generate a random NumPy array of shape (30,) with values between 2000 and 15000
# We use 15001 as np.random.randint excludes the high value
step_counts_flat = np.random.randint(2000, 15001, size=(30,))

# Reshape the 1D array into a 5x6 matrix
step_counts_matrix = step_counts_flat.reshape(5, 6)

print(f"Step counts matrix (5x6):\n{step_counts_matrix}\n")

# Find:

# Overall average steps
# The singular "Weekly average steps" likely refers to the overall average over the weeks
overall_avg_steps = np.mean(step_counts_flat)
print(f"Overall average steps: {overall_avg_steps:.2f}\n")

# Day with maximum steps (1-based index)
# np.argmax finds the index of the max value in the flattened array
max_day_index_flat = np.argmax(step_counts_flat)
max_day_number = max_day_index_flat + 1
print(f"Day with maximum steps (1-based index): {max_day_number}\n")

# Number of days user crossed 10,000 steps
days_crossed_10k = np.sum(step_counts_flat > 10000)
print(f"Number of days user crossed 10,000 steps: {days_crossed_10k}\n")
```

✓ 0.0s

```
Step counts matrix (5x6):
[[ 2853  2198 13059  6500  3775 12780]
 [ 8738 11632 14121 14147  9569  9685]
 [12237  8979 13440  9450  5006 10111]
 [ 7795  6267 11129  8234  2081  5377]
 [ 3698 12583  3523  5875  8671 11832]]

Overall average steps: 8511.50

Day with maximum steps (1-based index): 10

Number of days user crossed 10,000 steps: 11
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
    - Compute daily average
    - Identify abnormal readings > 120
- For all patients:
    - Compute overall maximum and minimum

Code:

```
#Step 1: Initialize the NumPy array
import numpy as np
heart_rates = np.random.randint(60, 150, (10, 4))
print(f"Generated Heart Rate Array:\n{heart_rates}\n")
```

✓ 0.0s

```
Generated Heart Rate Array:
[[129 103 102  94]
 [ 84 122  83 126]
 [109  65 138 138]
 [103  85  88 115]
 [ 83 137 132 116]
 [104  94  92 129]
 [119  69  86  68]
 [133  68  93  96]
 [ 93 109 131  85]
 [ 86  85 125  68]]
```

```python
#Step 2: Compute daily averages and identify abnormal readings
print("--- Results Per Patient ---")
for i, rates in enumerate(heart_rates):
    avg = np.mean(rates)
    abnormal = rates[rates > 120]
    print(f"Patient {i+1}:")
    print(f"  Daily Average BPM: {avg:.2f}")
    print(f"  Abnormal Readings (>120 BPM): {abnormal if abnormal.size > 0 else 'None'}")
    print("-" * 20)
```

✓ 0.0s

```
--- Results Per Patient ---
Patient 1:
  Daily Average BPM: 107.00
  Abnormal Readings (>120 BPM): [129]
--------------------
Patient 2:
  Daily Average BPM: 103.75
  Abnormal Readings (>120 BPM): [122 126]
--------------------
Patient 3:
  Daily Average BPM: 112.50
  Abnormal Readings (>120 BPM): [138 138]
--------------------
Patient 4:
  Daily Average BPM: 97.75
  Abnormal Readings (>120 BPM): None
--------------------
Patient 5:
  Daily Average BPM: 117.00
  Abnormal Readings (>120 BPM): [137 132]
--------------------
Patient 6:
  Daily Average BPM: 104.75
  Abnormal Readings (>120 BPM): [129]
--------------------
...
Patient 10:
  Daily Average BPM: 91.00
  Abnormal Readings (>120 BPM): [125]
--------------------
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
#Step 3: Compute overall maximum and minimum
overall_max = np.max(heart_rates)
overall_min = np.min(heart_rates)
print("--- Overall Results ---")
print(f"Overall Maximum Heart Rate: {overall_max} BPM")
print(f"Overall Minimum Heart Rate: {overall_min} BPM")
```

✓ 0.0s

```
--- Overall Results ---
Overall Maximum Heart Rate: 138 BPM
Overall Minimum Heart Rate: 65 BPM
```

Question 6: Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- Compute difference between branches
- Calculate:
    - Average monthly sales of each branch
    - Which branch performed better overall?
- Compute **correlation** between A and B

Code:

```
import numpy as np

# Monthly sales data in thousands
branch_a = [122, 135, 150, 160, 155, 140]
branch_b = [110, 130, 148, 165, 158, 145]

# 1. Compute the month-by-month difference between branches (Branch A - Branch B)
# This assumes the lists are of the same length and order matters
sales_difference = [a - b for a, b in zip(branch_a, branch_b)]

# 2. Calculate average monthly sales of each branch
average_a = np.mean(branch_a)
average_b = np.mean(branch_b)
```

```
# 3. Compute the correlation between A and B
# np.corrcoef returns a correlation matrix, the value at [0, 1] is the coefficient
correlation_matrix = np.corrcoef(branch_a, branch_b)
correlation_coefficient = correlation_matrix[0, 1]

# Display results
print(f"Branch A Sales (in thousands): {branch_a}")
print(f"Branch B Sales (in thousands): {branch_b}")
print("-" * 40)
print(f"Monthly Sales Difference (A - B): {sales_difference} (in thousands)")
print(f"Average Monthly Sales for Branch A: ${average_a:.2f}k")
print(f"Average Monthly Sales for Branch B: ${average_b:.2f}k")
print(f"Correlation Coefficient between A and B: {correlation_coefficient:.4f}")
print("-" * 40)
```

```python
# 4. Determine which branch performed better overall
if average_a > average_b:
    print(f"Overall, Branch A performed better with higher average sales.")
elif average_b > average_a:
    print(f"Overall, Branch B performed better with higher average sales.")
else:
    print(f"Branch A and Branch B performed equally well on average.")
```

✓ 0.0s

```
Branch A Sales (in thousands): [122, 135, 150, 160, 155, 140]
Branch B Sales (in thousands): [110, 130, 148, 165, 158, 145]
----------------------------------------
Monthly Sales Difference (A - B): [12, 5, 2, -5, -3, -5] (in thousands)
Average Monthly Sales for Branch A: $143.67k
Average Monthly Sales for Branch B: $142.67k
Correlation Coefficient between A and B: 0.9812
----------------------------------------
Overall, Branch A performed better with higher average sales.
```

Question 7: Given a 4×4 grayscale image:

[[100, 120, 130, 90],

[80 , 110, 140, 100],

[70 , 90 , 120, 130],

[110, 140, 150, 160]]

Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: new_pixel = pixel * 1.2
- Clip all values > 255
- Compute average brightness before & after transformation

Code:

```python
import numpy as np

# Original image data
image_data = [[100, 120, 130, 90],
              [80, 110, 140, 100],
              [70, 90, 120, 130],
              [110, 140, 150, 160]]
```

```python
# Convert to NumPy array
img_array = np.array(image_data, dtype=np.float64)

# --- Task related calculations ---

# Compute average brightness before transformation
avg_brightness_before = np.mean(img_array)
print(f"Average brightness before: {avg_brightness_before}")

# Normalize pixel values (0-1)
normalized_img = img_array / 255.0
# print(f"Normalized image array: \n{normalized_img}") # Optional print

# Apply a transformation: new_pixel = pixel * 1.2 on original scale data
transformed_img_array = img_array * 1.2

# Clip all values > 255 (limits values to the 0-255 range)
clipped_img_array = np.clip(transformed_img_array, 0, 255)

# Compute average brightness after transformation (on clipped array)
avg_brightness_after = np.mean(clipped_img_array)
print(f"Average brightness after: {avg_brightness_after}")

# Final Averages:
# Average brightness before: 115.0
# Average brightness after: 138.0
```

✓ 0.0s

```
Average brightness before: 115.0
Average brightness after: 138.0
```

Question 8: Dataset contains 100 samples with 4 features each.

Task:

- Generate a **100×4** matrix with values 1–100
- Standardize the dataset using:
- x_scaled = (x - mean) / std
- Perform:
    - Column-wise mean (should be ≈ 0)
    - Column-wise variance (should be ≈ 1)
- Verify results using NumPy functions

Code:

```python
import numpy as np

# 1. Generate a 100x4 matrix with values 1-100
# Reshaping a sequence of 400 values to fit a 100x4 matrix
data = np.linspace(1, 100, 400).reshape(100, 4)

# 2. Standardize the dataset: x_scaled = (x - mean) / std
# axis=0 ensures we calculate mean and std for each feature (column)
mean = np.mean(data, axis=0)
std = np.std(data, axis=0)
x_scaled = (data - mean) / std

# 3. Perform and verify results
col_mean = np.mean(x_scaled, axis=0)
col_var = np.var(x_scaled, axis=0)

print(f"Column-wise Mean (Expected ≈ 0): \n{col_mean}")
print(f"Column-wise Variance (Expected ≈ 1): \n{col_var}")

# Verification using NumPy functions
np.testing.assert_allclose(col_mean, 0, atol=1e-15)
np.testing.assert_allclose(col_var, 1, atol=1e-15)
print("\nVerification Successful: Results match expectations.")
```

✓ 0.0s

```
Column-wise Mean (Expected ≈ 0):
[ 1.37667655e-16  2.35367281e-16  2.39808173e-16 -3.24185123e-16]
Column-wise Variance (Expected ≈ 1):
[1. 1. 1. 1.]

Verification Successful: Results match expectations.
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the **busiest sensor** (highest total density)
- Apply a threshold:
    - Replace all values above 50 with 50 (traffic cap)
- Compute energy usage:

energy = Σ(sensor_density × 0.5)

code:

```python
import numpy as np

# Set a random seed for reproducibility
np.random.seed(42)

# Generate array shape (24, 6) with values 10-60 cars/min
density_data = np.random.randint(10, 61, size=(24, 6))

# Find hourly average
hourly_average = np.mean(density_data, axis=1)
# print(f"Hourly averages: \n{hourly_average}") # Optional print

# Identify the busiest sensor
total_density_per_sensor = np.sum(density_data, axis=0)
busiest_sensor_index = np.argmax(total_density_per_sensor)
# Using 1-based indexing for user clarity
busiest_sensor_id = busiest_sensor_index + 1
busiest_sensor_total = total_density_per_sensor[busiest_sensor_index]

print(f"Busiest sensor is Sensor {busiest_sensor_id} with total density {busiest_sensor_total}")

# Apply a threshold (traffic cap)
capped_density_data = np.clip(density_data, None, 50)
# print(f"Capped data: \n{capped_density_data}") # Optional print

# Compute energy usage: energy = Σ(sensor_density × 0.5)
energy_usage = np.sum(capped_density_data * 0.5)
print(f"Total energy usage: {energy_usage}")

# Final Results (from print output):
# Busiest sensor is Sensor 2 with total density 885
# Total energy usage: 2401.5
```

✓ 0.0s

```
Busiest sensor is Sensor 2 with total density 885
Total energy usage: 2401.5
```

Question 10: You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

F = [[5,2,1],

[3,1,4],

[6,3,2],

[4,2,5],

[7,1,3]]

Transformation matrix:

T = [[2,1,0],

[1,3,1],

[0,2,2]]

Task:

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

Code:

```
import numpy as np

# Force vectors applied to 5 components
F = np.array([
    [5, 2, 1],
    [3, 1, 4],
    [6, 3, 2],
    [4, 2, 5],
    [7, 1, 3]
])
```

```
# Transformation matrix
T = np.array([
    [2, 1, 0],
    [1, 3, 1],
    [0, 2, 2]
])

# Compute transformed forces: F_new = F x T
F_new = F @ T
print(f"Transformed Forces:\n{F_new}")

# Compute magnitude of each transformed force vector
magnitudes = np.linalg.norm(F_new, axis=1)
print(f"Magnitudes:\n{magnitudes}")
```

```python
# Identify the component experiencing highest transformed force
highest_force_index = np.argmax(magnitudes)
highest_force_magnitude = magnitudes[highest_force_index]
highest_force_component_id = highest_force_index + 1
print(f"Highest Force Component: {highest_force_component_id}, Magnitude: {highest_force_magnitude}")

# Perform eigenvalue decomposition of T
eigenvalues, eigenvectors = np.linalg.eig(T)
print(f"Eigenvalues of T:\n{eigenvalues}")
print(f"Eigenvectors of T:\n{eigenvectors}")
```

✓ 0.0s

```
Transformed Forces:
[[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
Magnitudes:
[18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
Highest Force Component: 4, Magnitude: 25.37715508089904
Eigenvalues of T:
[4.30277564 2.         0.69722436]
Eigenvectors of T:
[[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.70737023e-17 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

LAB 04: Seaborn and scikit learn

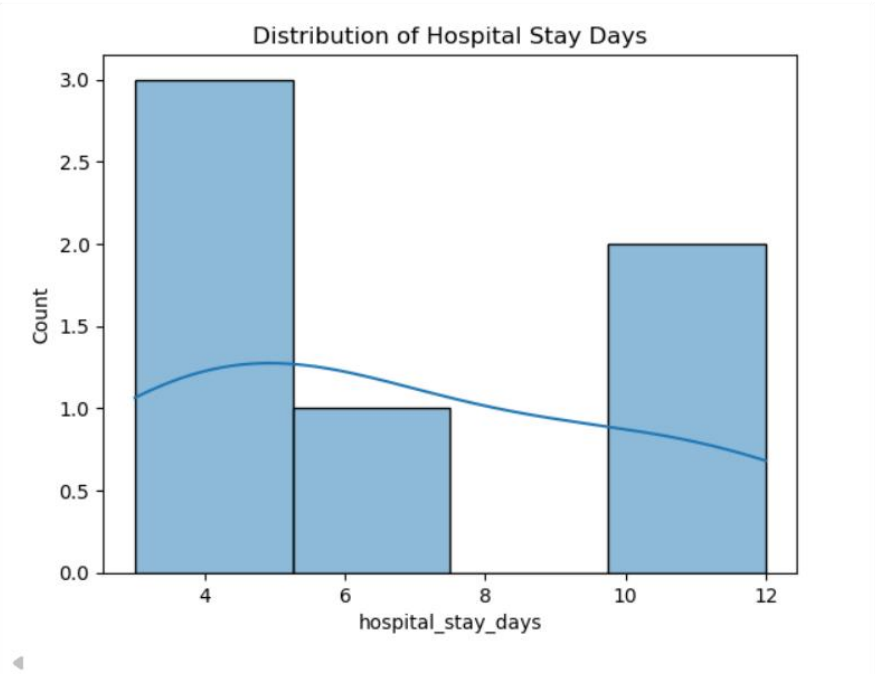
Q1. Patient Stay Analysis (Healthcare)

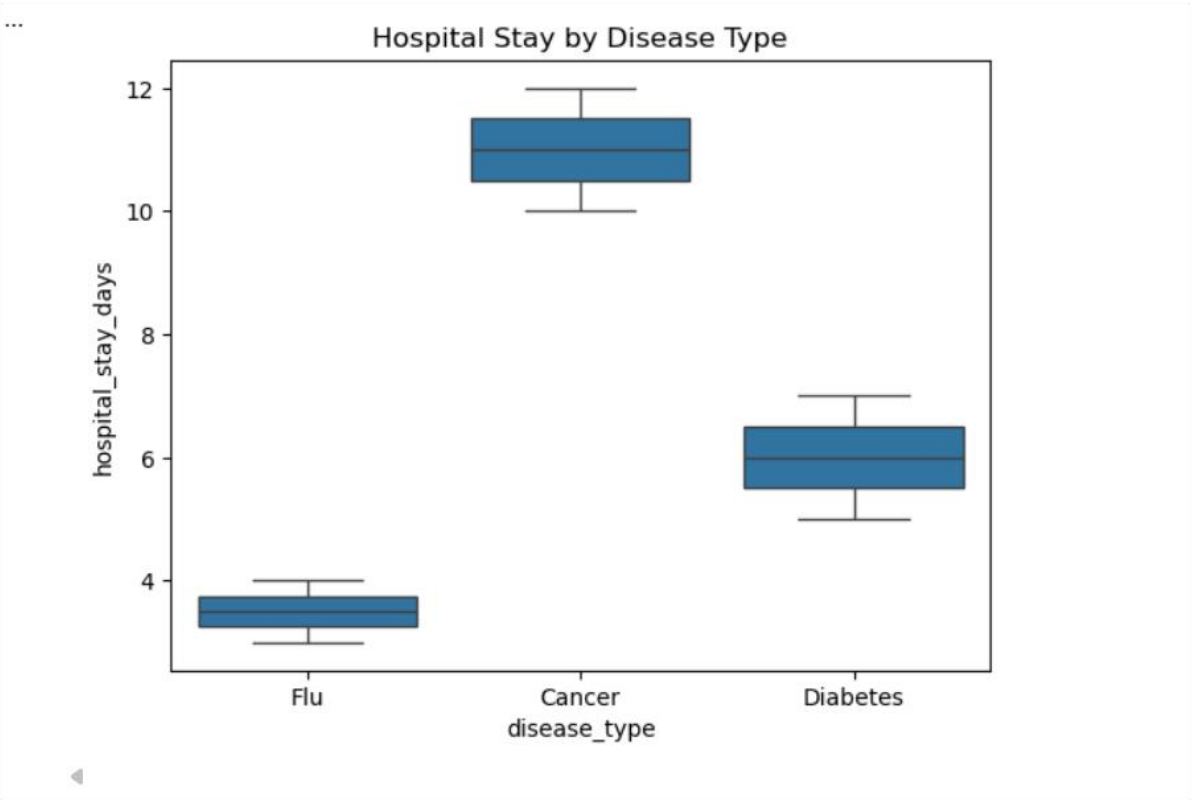
You are given a dataset containing age, disease_type, and hospital_stay_days.
Write a Python program using **Seaborn** to:

- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    'age': [25, 40, 60, 30, 50, 70],
    'disease_type': ['Flu', 'Flu', 'Cancer', 'Diabetes', 'Cancer', 'Diabetes'],
    'hospital_stay_days': [3, 4, 10, 5, 12, 7]
})

sns.histplot(data['hospital_stay_days'], kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.show()

sns.boxplot(x='disease_type', y='hospital_stay_days', data=data)
plt.title("Hospital Stay by Disease Type")
plt.show()

print(data.groupby('disease_type')['hospital_stay_days'].median())
```

✓ 5.0s






```
disease_type
Cancer      11.0
Diabetes     6.0
Flu          3.5
Name: hospital_stay_days, dtype: float64
```

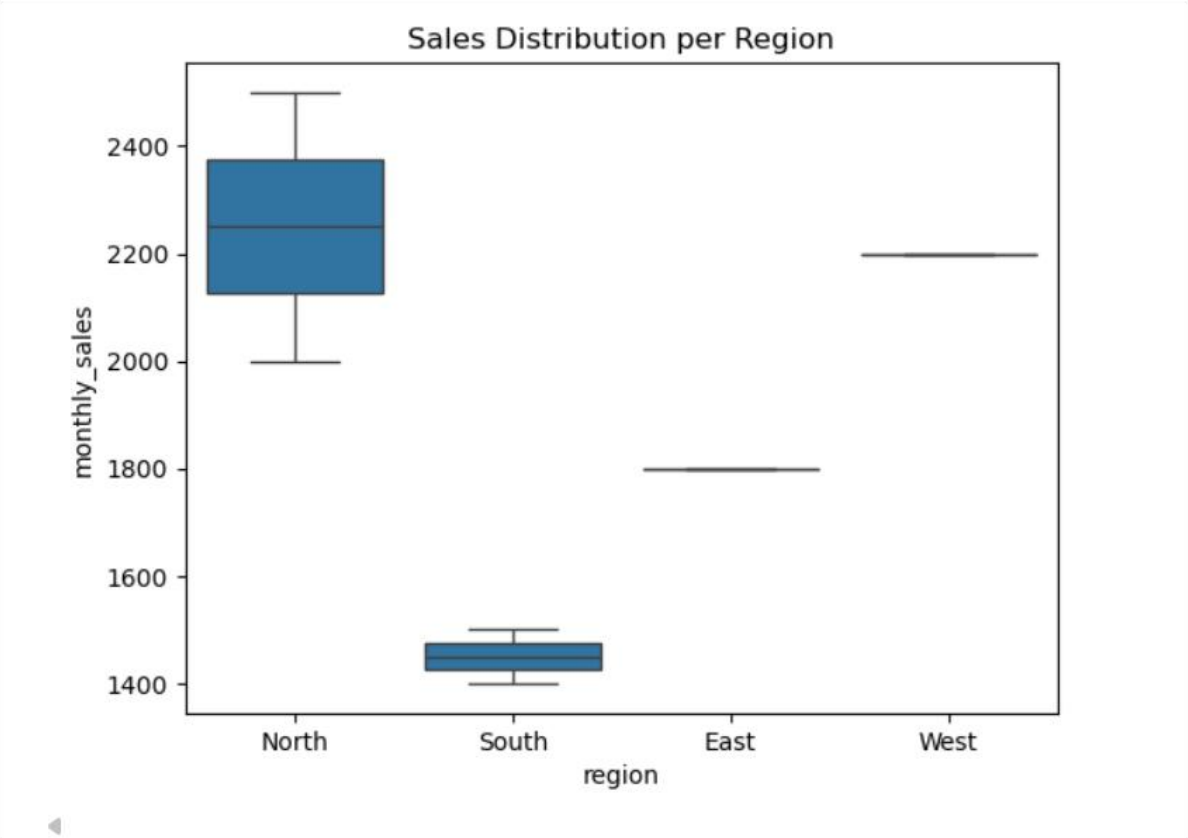
Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.
Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

```
data = pd.DataFrame({
    'region': ['North', 'South', 'East', 'West', 'North', 'South'],
    'monthly_sales': [2000, 1500, 1800, 2200, 2500, 1400]
})

sns.boxplot(x='region', y='monthly_sales', data=data)
plt.title("Sales Distribution per Region")
plt.show()

print(data.groupby('region')['monthly_sales'].std())
```

✓ 0.1s



```
...   region
      East            NaN
      North    353.553391
      South     70.710678
      West            NaN
      Name: monthly_sales, dtype: float64
```

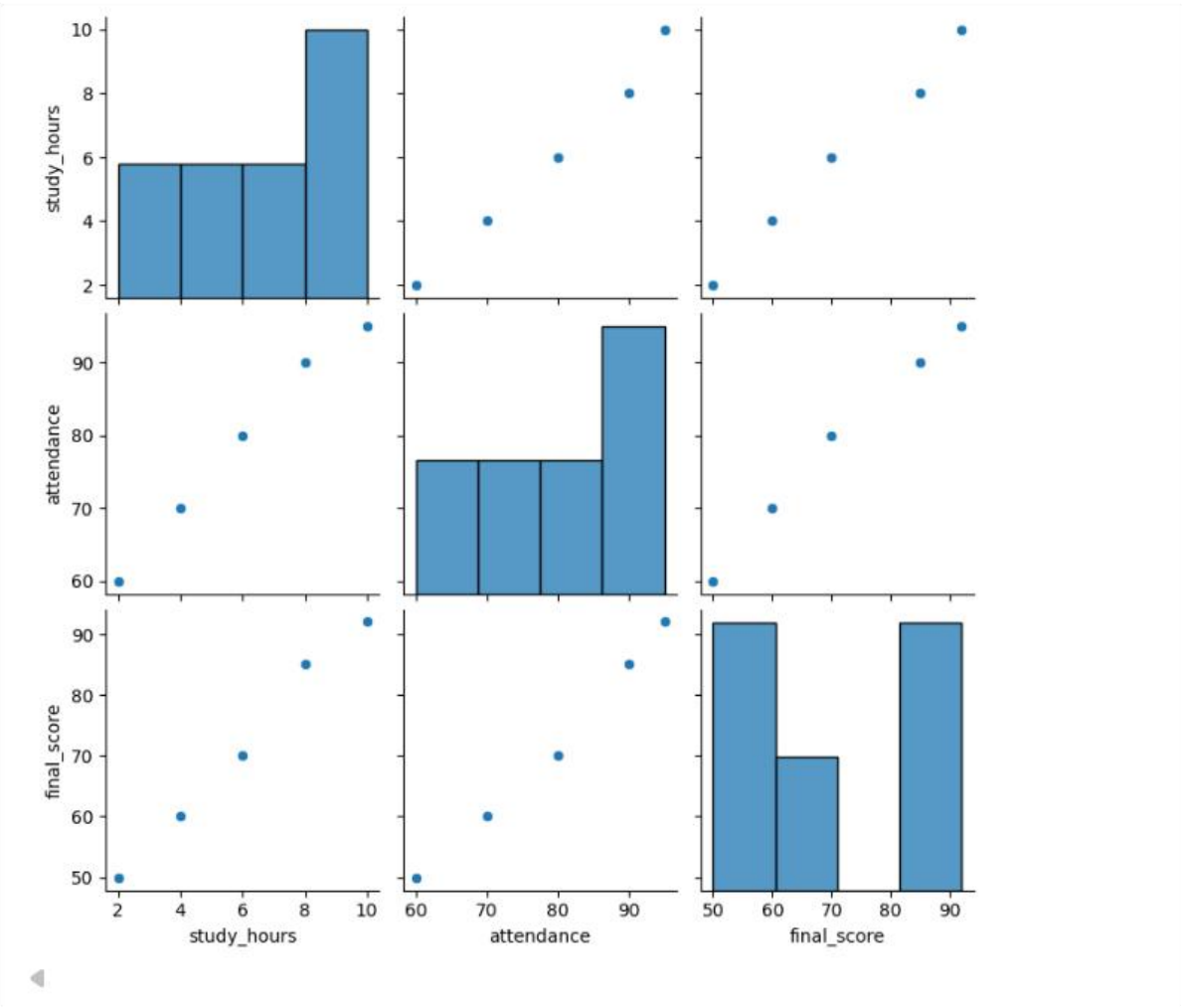
Q3. Student Performance Correlation (Education)

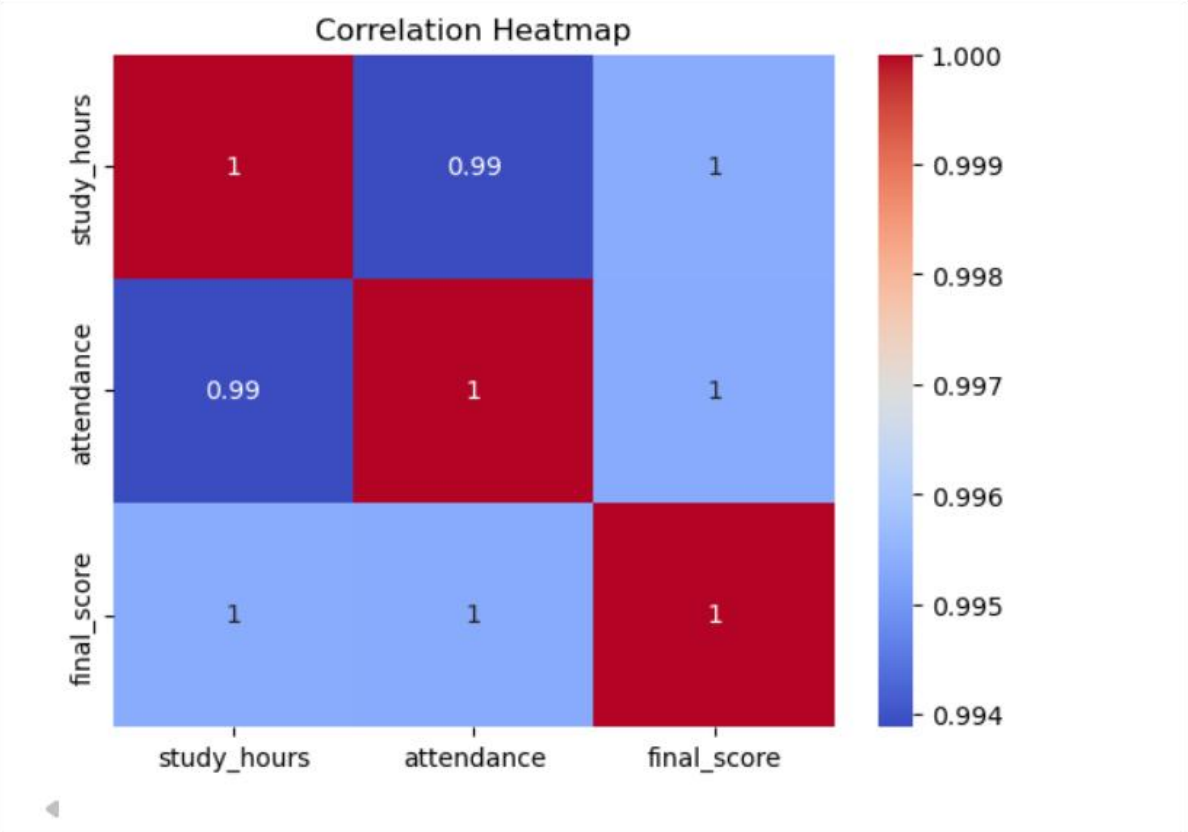
A dataset contains study_hours, attendance, and final_score .
Using Seaborn:

- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

```
data = pd.DataFrame({
    'study_hours': [2, 4, 6, 8, 10],
    'attendance': [60, 70, 80, 90, 95],
    'final_score': [50, 60, 70, 85, 92]
})

sns.pairplot(data)
plt.show()

sns.heatmap(data.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

✓ 1.1s


study_hours
attendance
final_score
2
4
6
8
10
60
70
80
90
50
60
70
80
90



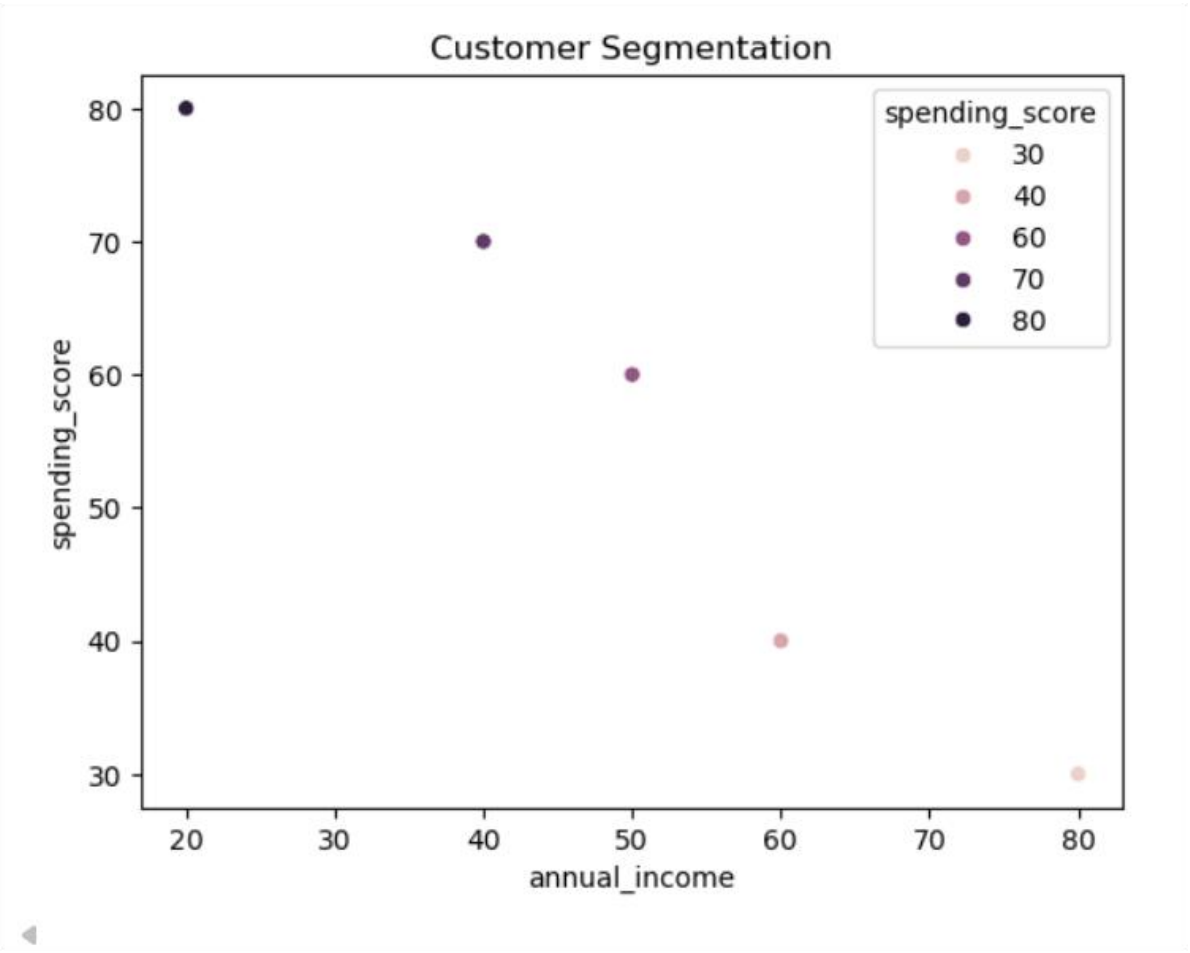
Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.
Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

```
data = pd.DataFrame({
    'age': [22, 35, 45, 30, 50],
    'annual_income': [20, 50, 60, 40, 80],
    'spending_score': [80, 60, 40, 70, 30]
})

sns.scatterplot(x='annual_income', y='spending_score',
                hue='spending_score', data=data)
plt.title("Customer Segmentation")
plt.show()
```

✓ 0.1s


Customer Segmentation
spending_score
30
40
60
70
80
spending_score
80
70
60
50
40
30
20
30
40
50
60
70
80
annual_income

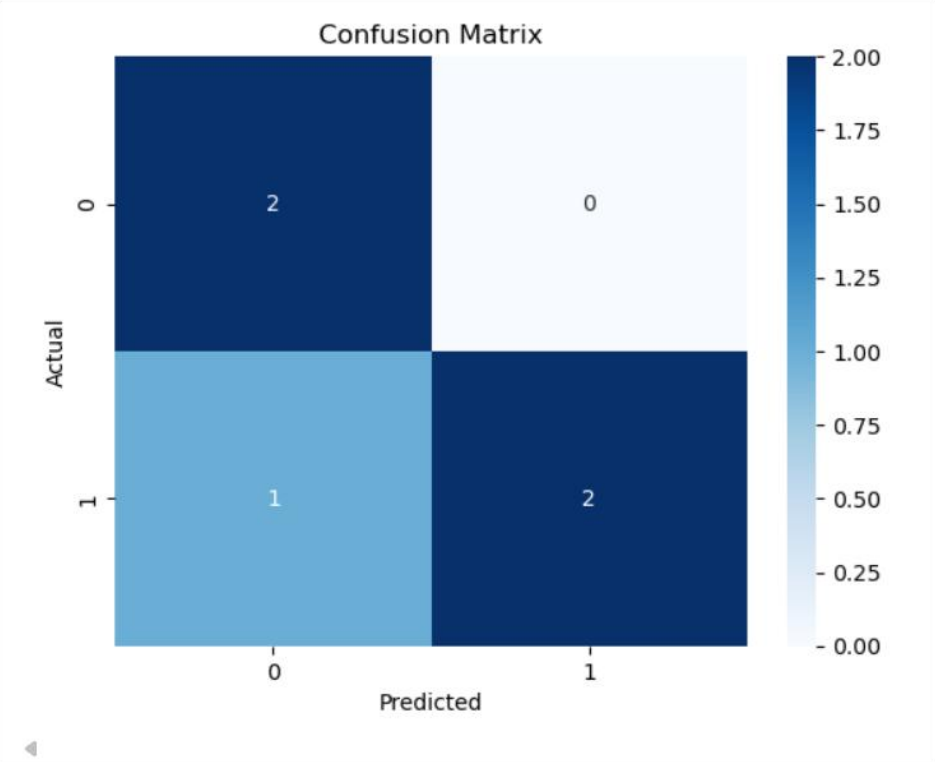
Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.
Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

```
from sklearn.metrics import confusion_matrix

y_true = [0, 1, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

✓ 0.3s



Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import numpy as np

data = pd.DataFrame({
    'tenure': [1, 12, 24, 5, 18],
    'monthly_charges': [50, 70, 90, 60, 80],
    'contract_type': ['Month-to-month', 'One year', 'Two year', 'Month-to-month', 'One year'],
    'churn': [1, 0, 0, 1, 0]
})

le = LabelEncoder()
data['contract_type'] = le.fit_transform(data['contract_type'])

X = data.drop('churn', axis=1)
y = data['churn']

X_train, X_test, y_train, y_test = train_test_split(X, y)

model = LogisticRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

✓ 0.2s

```
Accuracy: 1.0
```

Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.
Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = pd.DataFrame({
    'area': [1000, 1500, 2000, 1200, 1800],
    'bedrooms': [2, 3, 4, 2, 3],
    'location_score': [6, 7, 9, 6, 8],
    'price': [100000, 150000, 220000, 120000, 180000]
})

X = data[['area', 'bedrooms', 'location_score']]
y = data['price']

X_train, X_test, y_train, y_test = train_test_split(X, y)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, pred))
```

✓ 0.0s

```
MSE: 199995384.68934852
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.
Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

```
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

data = pd.DataFrame({
    'glucose': [120, 150, 100, 180, 140],
    'BMI': [25, 30, 22, 35, 28],
    'age': [25, 45, 30, 50, 40],
    'outcome': [0, 1, 0, 1, 1]
})

X = data.drop('outcome', axis=1)
y = data['outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = DecisionTreeClassifier()
model.fit(X_scaled, y)

y_pred = model.predict(X_scaled)
print(classification_report(y, y_pred))
```

✓ 0.2s

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         2
           1       1.00      1.00      1.00         3

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

Q4. Email Spam Detection

You are given email text and spam labels.
Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

emails = ["Win money now", "Meeting tomorrow", "Free prize", "Project update"]
labels = [1, 0, 1, 0]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

test_email = ["Free money offer"]
test_vec = vectorizer.transform(test_email)
print("Spam Prediction:", model.predict(test_vec))
```

✓ 0.0s

```
Spam Prediction: [1]
```

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.
Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

```
from sklearn.ensemble import RandomForestClassifier

data = pd.DataFrame({
    'attendance': [60, 80, 50, 90, 70],
    'mid_marks': [40, 70, 35, 85, 60],
    'final_marks': [45, 75, 40, 90, 65],
    'risk': [1, 0, 1, 0, 0]
})

X = data.drop('risk', axis=1)
y = data['risk']

model = RandomForestClassifier()
model.fit(X, y)

print("Feature Importance:", model.feature_importances_)
```

✓ 0.3s

```
Feature Importance: [0.29411765 0.35294118 0.35294118]
```

LAB 05: Project Development

Learning Objectives / Outcomes:

- To understand the **TensorFlow library**, its purpose in machine learning and deep learning, and its key features such as multi-platform support, scalability, and visualization with TensorBoard.
- To learn the concept of **tensors, operations, and computational graphs**, and how TensorFlow executes computations using eager execution or sessions.

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Why is it called TensorFlow?

- **Tensor** → a fancy name for multi-dimensional data (like arrays, matrices).
- **Flow** → means data flows through different layers of a neural network.

So **TensorFlow = data (tensors) flowing through operations**.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis

- Recommendation systems (like Netflix)
- Self-driving cars

```python
import tensorflow as tf
from tensorflow import keras

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Dummy data
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

# Train the model
model.fit(x, y, epochs=50)

# Predict
print(model.predict([5]))
```

2. How TensorFlow Works

TensorFlow works with **tensors**, which are just **multi-dimensional arrays**. It builds a **computation graph**, where each node represents a mathematical operation, and edges represent data (tensors) flowing between operations.

Key Components:

1. **Tensors:** Basic data structure (like arrays or matrices).
   Example: Scalars (0D), Vectors (1D), Matrices (2D), etc.
2. **Operations (Ops):** Mathematical operations applied to tensors (like addition, multiplication).
3. **Computational Graph:** A graph representing the flow of data through operations.

4. **Sessions (TF1.x) / Eager Execution (TF2.x):**
    - TensorFlow 1.x used sessions to run graphs.
    - TensorFlow 2.x executes operations immediately (like regular Python), making it easier to debug

Implementation

```
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',               # Stochastic Gradient Descent optimizer
    loss='mean_squared_error'      # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

Output

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539  13.9809065 15.975274 ]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```